



Audit Logs: Event Schema & Sanitized Examples

Overview

The MTPI/LambdaProof system uses a layered audit architecture to ensure all state transitions, decisions, and identity operations are verifiable while preserving user privacy. Audit events are emitted both on-chain (via Solidity contract events) and off-chain (via LambdaTrace objects logged to `logs/audittrail.log`). All events use **hash-only** semantics—no PII or sensitive data ever appears on-chain.^{[1][2]}

On-Chain Event Schema

Core State Transition Events

Event	Indexed Fields	Non-Indexed Fields	Purpose
XiStateTransition	previousState, newState, executor	timestamp, proofHash	Records quantum state transitions with zk-proof verification ^[3]
PrimeGateActivation	primeIndex, activator	proofHash	Logs prime gate activations with monotonic ordering ^[3]
IdentityRegistered	entity	identityHash	Records new identity registrations ^[3]

RecoveryModeActivated	—	timestamp	Signals CSL-compliant silent recovery entry ^[3]
LTraceLinked	ltraceHash, actor	tokenId	Associates Λ -Trace decisions with on-chain tokens ^[1]

Event Field Definitions

```
// E-License v1.2 Event Schema
event XiStateTransition(
    bytes32 indexed previousState, // Poseidon hash of prior state
    bytes32 indexed newState,      // Poseidon hash of new state
    address indexed executor,      // Transaction sender (not identity)
    uint256 timestamp,            // Block timestamp
    bytes32 proofHash             // BLAKE3 hash of zk-SNARK proof
);

event PrimeGateActivation(
    uint256 indexed primeIndex,    // Activated prime (validated via Miller-Rabin)
    address indexed activator,     // Actor triggering activation
    bytes32 proofHash             // Proof hash for audit linkage
);

event IdentityRegistered(
    address indexed entity,        // Wallet address (transient)
    bytes32 identityHash          // Poseidon(primeIdHash) - permanent identity
);
```

Off-Chain Λ -Trace Audit Schema

The Λ -Trace layer provides the canonical decision/audit format for all evaluations, binding off-chain decisions to on-chain state.^[1]

Core Λ -Trace Fields

Field	Type	Description
-------	------	-------------

traceId	string	UUID or hash-derived identifier
workHash	bytes32	Hash of evaluated object (proposal, asset, etc.)
primeIdHashOwner	bytes32	Poseidon-hashed prime identity of owner
evaluatorId	string	DID, address, or node ID of evaluator
scores	object	Structured scores (0–10000 basis points)
ethicsPass	boolean	Primary decision flag
mode	enum	tribunal silent supervised
engineVersion	string	Evaluation engine identifier (e.g., qcalc-v1)
evidenceCid	string	IPFS/Arweave CID for full trace & evidence
timestamp	uint64	Unix timestamp
signatures	object	evaluatorSig, optional engineSig

Canonicalization & Hashing

All Λ-Trace objects undergo deterministic serialization (stable key ordering, no floating-point ambiguity), then are hashed via keccak256 to produce a single `1traceHash`. This hash serves as the **sole canonical identifier** across all on-chain and off-chain references.^[1]

Sanitized Examples

Example 1: Quantum State Transition Event

```
{  
  "event": "XiStateTransition",
```

```

    "blockNumber": 19847293,
    "transactionHash": "0x7a3f...b92e",
    "args": {
      "previousState":
"0x1a2b3c4d5e6f708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6e7f801",
      "newState": "0x2b3c4d5e6f708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6e7f80192",
      "executor": "0x9F3a...2eC1",
      "timestamp": 1736380800,
      "proofHash": "0x8d9e0f1a2b3c4d5e6f708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6"
    },
    "driftCompliant": true,
    "driftValue": "0.18"
  }
}

```

Example 2: Prime Gate Activation Event

```

{
  "event": "PrimeGateActivation",
  "blockNumber": 19847301,
  "transactionHash": "0x4e5f...a1b2",
  "args": {
    "primeIndex": 17,
    "activator": "0x9F3a...2eC1",
    "proofHash": "0xf0e1d2c3b4a5968778695a4b3c2d1e0f1a2b3c4d5e6f708192a3b4c5d6e7f801"
  },
  "primeValidation": "CERTIFIED",
  "previousPrimeIndex": 13
}

```

Example 3: Off-Chain Λ -Trace Decision Record

```

{
  "traceId": "ltrace-8f2a9b4c-1d3e-5f6a-7b8c-9d0e1f2a3b4c",
  "workHash": "0x3c4d5e6f708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6e7f8019203",
  "primeIdHashOwner":
"0x5e6f708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6e7f801920304a5",
  "evaluatorId": "did:mtpi:evaluator:node-07",
  "scores": {
    "noveltyBps": 8200,

```

```

    "feasibilityBps": 7400,
    "safetyBps": 9100,
    "ethicsBps": 8800,
    "impactBps": 7900
  },
  "ethicsPass": true,
  "mode": "supervised",
  "engineVersion": "latrace-v0.2",
  "evidenceCid": "bafybeig...xyz",
  "timestamp": 1736380856,
  "signatures": {
    "evaluatorSig": "0xa1b2c3d4...8f9e",
    "engineSig": null
  },
  "ltraceHash": "0x6f708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c506"
}

```

Example 4: CSL Certificate Log Entry

```

{
  "certificateType": "CSP",
  "timestamp": 1736380812,
  "slopeUpperBound": 0.94,
  "slopeTotalUpperBound": 0.97,
  "gapLowerBound": 0.03,
  "projectorFlags": {
    "nonExpansive": true,
    "lipschitzBound": 1.0
  },
  "residuals": [],
  "assumptions": ["affine-core", "contraction-safe"],
  "contextHash": "0x708192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d6e7f80192a3b4c5d607",
  "acceptanceStatus": "CERTIFIED",
  "proofLevel": "formal"
}

```

Audit Log File Format

Audit events are appended to `logs/audittrail.log` in newline-delimited JSON format:^[2]

```
{"timestamp":"2026-01-08T18:32:15Z","event":"XiStateTransition","txHash":"0x7a3f...","status":"success"}
{"timestamp":"2026-01-08T18:32:22Z","event":"PrimeGateActivation","primeIndex":17,"status":"success"}
{"timestamp":"2026-01-08T18:33:01Z","event":"IdentityRegistered","idHashPrefix":"0x5e6f...","status":"success"}
```

Privacy & Compliance Guarantees

- **No PII on-chain:** Only cryptographic hashes and attestation UIDs appear in contracts^[1]
- **Drift compliance:** All transitions satisfy $\Delta\xi_t \leq 0.3$ ^[4]
- **Proof reuse prevention:** `usedProofHashes` mapping serves as a nullifier^[3]
- **Silence Clause adherence:** No contract requires ongoing activity solely because a token or Λ -Trace exists^[1]
- **Auditability:** Given any token or attestation, auditors can resolve to `ltraceHash`, fetch the full Λ -Trace JSON, verify signatures, and reconstruct the decision chain^[1]

**

-
1. 4.-L-trace-Overview.pdf
 2. MTPI_Instructions.pdf
 3. MTPI-RootContract.pdf
 4. LProof-IP-Patent-Research.pdf
 - a. Aproof × Mtpi Lawful-by-design Integration — Implementation Plan.pdf
 5. 2.ΛRootContract II & III - MTPI - NODE.js.pdf
 - b. Aproof Monorepo — Plain-language Overview V0.pdf

- c. Λ proof Banking Protocols.pdf
 - d. Λ proof Healthcare Protocols V0.1.pdf
 - e. Λ dapps On Web4 - Developer Overview.pdf
6. Building a Λ Dapp_ A Developer's Implementation Guide.pdf
 7. MTPI_RootContract_Project.pdf
 8. Patent Search Results.pdf
 9. The Λ Proof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf
 10. The Λ Proof Architecture_ Web4.pdf
 11. Verifiable, Privacy-Preserving Applications.pdf
 12. LProof-3of3.pdf
 13. Conscious_Sovereignty_Layer.pdf
 14. Milestone-Vision-Timeframe.csv
 15. Lets-further-develop-the-SAFE_-awesome-lets-turn.pdf
 16. Ξ Constitution.pdf
 17. Λ^P -Archivum.pdf